



M363 Unit 12

UNDERGRADUATE COMPUTING

Software engineering with objects



The case study: Part 3

Unit **12**

This publication forms part of an Open University course M363 *Software engineering with objects*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

Java and all Java-based trademarks are trademarks of Sun Microsystems, Inc.; Motif is a registered trademark of the Open Group; Rational Unified Process is a registered trademark of International Business Machines Corporation; Unified Modeling Language and UML are trademarks of Object Management Group, Inc. Design by Contract is a trademark of Interactive Software Engineering. Other product and company names may appear in the M363 course material. Rather than use a trademark symbol with every occurrence of a trademarked name, we use the names only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2008.

Copyright © 2008 The Open University.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by SR Nova Pvt. Ltd, Bangalore, India.

Printed in the United Kingdom by Thanet Press Ltd, Margate.

ISBN 978 0 7492 1618 4

1.1



CONTENTS

1	Introduction	5
2	Architecture	6
2.1	Summary of section	7
3	Implementing the model	8
3.1	Implementing the design class diagram	8
3.2	Implementing operations	21
3.3	More complex operations	24
3.4	Summary of section	36
4	The Façade pattern	37
4.1	Summary of section	38
5	The user interface	39
5.1	Handling the check of available rooms for the <i>make reservation</i> use case	43
5.2	Handling the <i>Confirm reservation</i> button of the <i>make reservation</i> use case	45
5.3	Handling when a name has been entered	47
5.4	Handling hotel selection	47
5.5	Handling clearing of data fields	48
5.6	Summary of section	48
6	Testing	49
6.1	Unit testing	49
6.2	System testing	50
6.3	Summary of section	51
7	Summary	52
	References	53
	Index	54

M363 COURSE TEAM

Robin Laney, Course Chair, Author and Academic Editor

Leonor Barroca, Author

Ralph Greenwell, Course Manager

Charles Haley, Author

Nigel Kermode, Critical Reader

Martin Shepperd, External Adviser, Brunel University

Richard Walker, Critical Reader

Andy Allum, Course Website Developer

Kim Dulson, Software Procurement

Phillip Howe, Media Assistant

Callum Lester, Software Developer

Tara Marshall, Print Procurement

Sandy Nicholson, Freelance Copy Editor

Andy Seddon, Media Project Manager

Lucinda Simpson, Editor

Sue Stavert, Technical Tester

Andrew Whitehead, Graphic Artist

Thanks are due to the Desktop Publishing Unit, Faculty of Mathematics and Computing.

1

Introduction

In this unit, we will consider the overall structure (or architecture) of the hotel system software, in particular, how the software that implements the model given in *Unit 8* fits with a user interface. We show how the system, including the user interface, is implemented, and finally outline a set of tests.

The software development activities are carried through for an initial partition of the whole system that was originally identified in *Unit 4*, containing the following four use cases:

- ▶ *make reservation;*
- ▶ *cancel reservation;*
- ▶ *check in guest;*
- ▶ *check out guest.*

For this partition, you will see only limited functionality being implemented in a first iteration. In particular, we will focus on success scenarios. Also, although we discuss distribution when considering the overall architecture, neither distribution nor concurrency will be implemented in this first iteration. By taking this approach, we can quickly produce an initial working system that can be used as a prototype to allow us to confirm some of the users' requirements and make refinements where appropriate. Such timely feedback allows for adjustments in subsequent iterations. Also, as this is a prototype, Design by Contract is not carried through to the implementation. You will therefore see a greatly simplified version of the system, but it will be detailed enough to highlight important concepts and problems in software development.

2 Architecture

Throughout the first iteration, we will keep the architecture of the final system in mind, to ensure that there is a smooth path through future iterations towards our goal.

Since our system is designed to be used across the hotel chain, the overall architecture will need to deal with distribution. To keep things relatively simple we will adopt a client–server architecture with a single server based at the hotel system's headquarters as shown in Figure 1. This architecture will support the provision of client applications running on PCs in particular hotels, as well as web clients running in web browsers, where potential customers make reservations over the internet. We assume that the PC clients in hotels connect to the server via the internet. Consistent with this client–server architecture, we can adopt a layered architecture where presentation issues (such as the user interface) are considered separately from the business logic. Persistence issues are outside the scope of this course; in principle they would fit into a separate persistence layer. While we are not addressing concurrency in this iteration, our business-model layer could adopt a component architecture, such as Enterprise JavaBeans, in order to deal with such issues.

Persistence is covered in a sister course, M362.

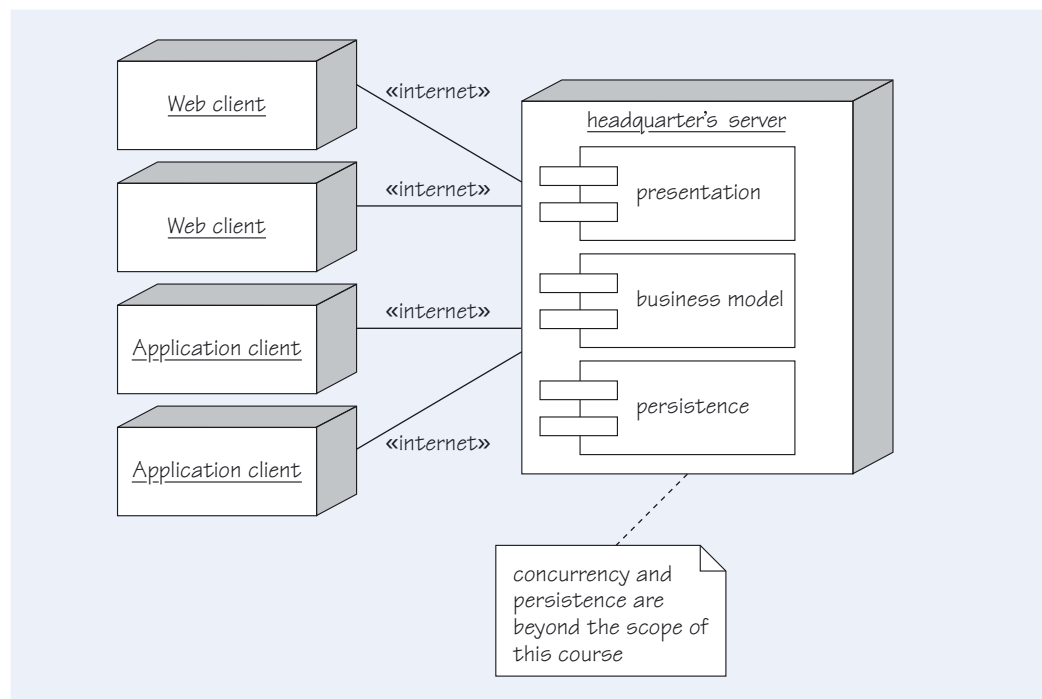


Figure 1 Deployment diagram

In order to meet the requirements of the current iteration while being consistent with the target architecture as described, we will adopt an initial architecture that splits the user interface of the system from the business model. We will use a Model–View–Controller architecture, but given the relative simplicity of the user interface needed for our application we will merge the view and controller. This is a very common approach and indeed is the approach that Swing, the Java user interface toolkit, takes. Our architecture for this iteration will therefore be a two-layered approach consisting of a user interface and a business model. We will collect all of

the components we develop in each of these layers into Java packages (one for the user interface and another for the model), in order to keep the structure of the code in line with the architecture.

Having decided on a two-layer architecture (for the user interface and the business model), we need to think carefully about the interface between the two layers. We want the user interface to have an interface to the model that is relatively simple but does not expose more of the model than is necessary. A simple interface allows work to be carried out on the user interface without detailed knowledge of the business model. Furthermore, by restricting the access the user interface has to the model, we can limit the scope for errors in the user interface to percolate into the model. For these reasons we will make use of the Façade pattern to achieve the two goals just described; this will be described in detail in Section 4. In the code we will add a package for the Façade. The architecture is shown in diagrammatic form in Figure 2.

A Façade provides a unified higher-level interface to a set of interfaces (see E. Gamma et al. [1995]).

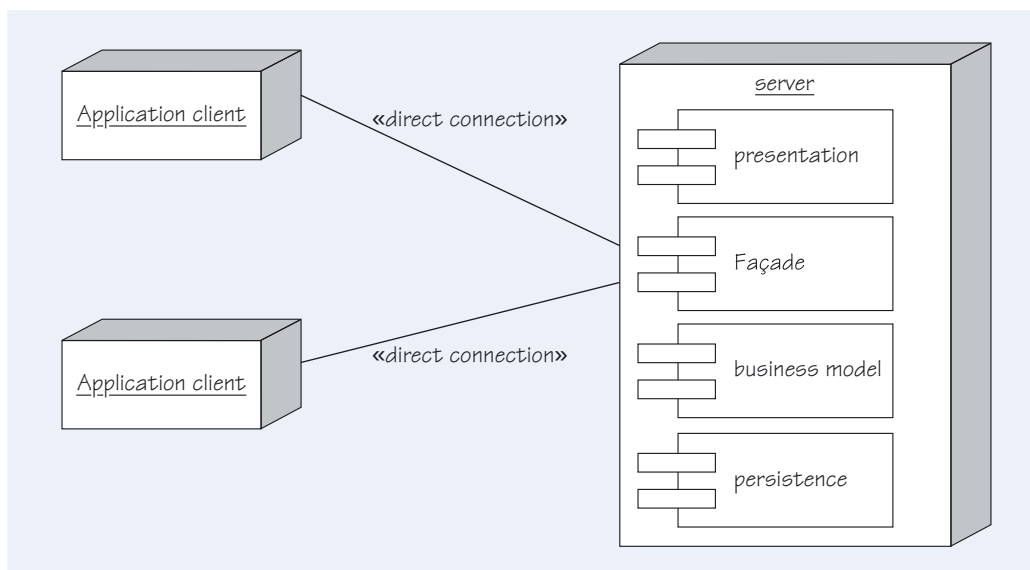


Figure 2 Simplified deployment diagram showing the architectural overview

2.1 Summary of section

The architecture adopted for the hotel system is a client-server architecture with a single server based at the hotel chain's headquarters. A layered architecture separates presentation issues from the business logic; this is a two-layer Model-View-Controller architecture, with the view and controller merged. The presentation has a simple interface to the model that uses the Façade pattern to avoid exposing more of the model than is necessary.

3

Implementing the model

In this section we implement the revised class diagram as it was drawn at the end of *Unit 8*. As we consider how to derive code from the design model, we will refer back to the software requirements and the modelling done in *Unit 4* as necessary, in order to implement the code and to verify the implementation.

To be able to realise a system design in a programming language, all the elements of the model need to be expressed in that language. In this unit we look at some examples of how this can be done in Java, with reference to the design model for the hotel chain system.

At this point you should open the code for the hotel chain system and familiarise yourself with it. We will be looking at the implementation of different parts of the design model, and you will be asked to write bits of code on paper. Try to do those exercises without looking at the code, but check with the code once you have done them. Please note that comments in the text and in the code are not always consistent; the comments in the text are designed to illustrate specific teaching points. You should also be aware that code evolves and changes throughout the unit and is not, therefore, always consistent with the prototype's code.

3.1 Implementing the design class diagram

At the end of *Unit 8* we recorded design decisions in a design class diagram, repeated here in Figure 3. These design decisions concern the allocation of operations to classes, the direction of navigation of associations, and the introduction of qualified associations. Now we proceed to create a basic class definition in Java for that design class diagram.

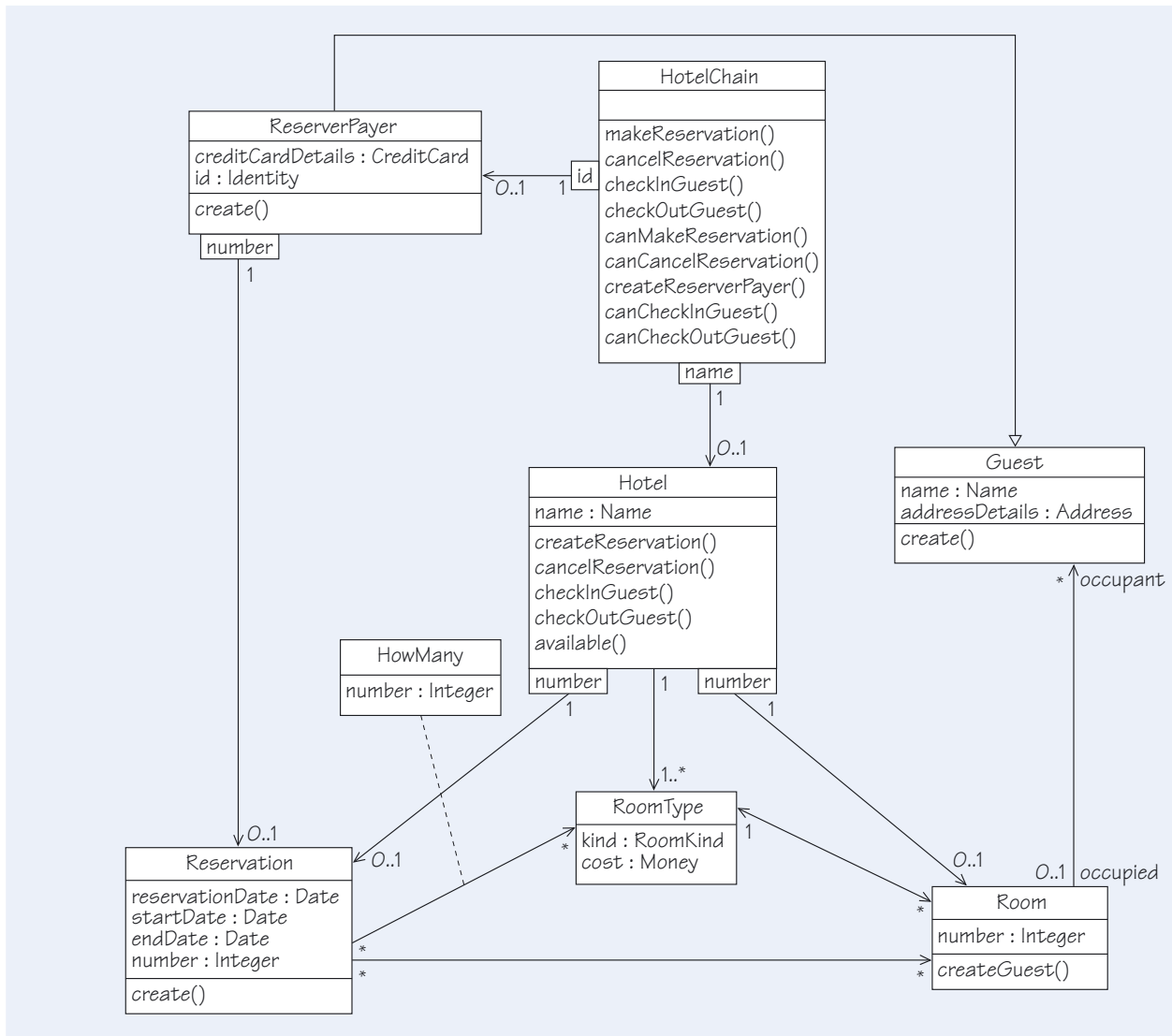


Figure 3 Design class diagram

Classes

In the following sections we look at how to implement classes, attributes and associations. We begin by reviewing some key ideas which we need to use. Many of these will probably be familiar to you already.

In Java a class or an interface always belongs to some package. If no package is explicitly declared, the class or interface will be placed in an unnamed default package. Package declarations serve two main purposes in Java. First, they allow for a logical organisation of classes and the establishment of visibility rules: classes (and their attributes and operations) can be declared to be visible within their own package or across packages (we return to visibility in Java in the following sections). Second, packages help resolve name conflicts: a class is identified not just by its name, but also by the name of the package to which it belongs.

We create a package for the hotel system and proceed to declare the classes within this package.

```
package hotel.system;
// class declaration
public class HotelChain {
    //...
}
//...
```

Inheritance

An object-oriented language allows the programmer to declare an inheritance relationship between classes. For instance, the relationship between `Guest` and its subclass `ReserverPayer` would be expressed in Java by using the `extends` keyword in the class declaration.

Note that Java supports only single inheritance; that is to say, a class can inherit directly from one superclass at most.

```
// inheritance
public class ReserverPayer extends Guest {
    //...
}
```

Inheritance is one of the main mechanisms for the reuse of code in an object-oriented language. The classes you develop will in one way or another inherit from the existing library of the language you are using.

Attributes

Attributes are realised in a programming language as **variables** within a type declaration. Variables can be either:

- ▶ **instance variables**, which correspond to object attributes, that is, attributes associated with each instance of the class; or
- ▶ **class variables**, which correspond to class attributes, that is, attributes associated with the class itself. Class variables are shared by the class itself and all the (possibly zero) instances; they are created when the class is initialised.

In Java, variables are declared as class variables by being labelled with the keyword `static`. In order to declare variables, their visibility and type must be known. In Java, the visibility of both classes and variables can be declared, using the keywords `private`, `public` and `protected`. A `public` class is visible everywhere to all classes; if a class has no modifier, it is visible only within its own package (that is, it is `private` by default). For a class member, you can also use the `public` modifier when it is visible outside its class or the `private` modifier when it can only be accessed in its own class. The `protected` modifier specifies that the member can only be accessed within its own package and, in addition, by a subclass of its class in another package. Omitting the visibility keyword restricts access to its own package.

A variable declaration uses the following syntax:

```
visibilityKeyword variableType variableName;
```

On the other hand a class may implement any number of interfaces.

<http://java.sun.com/docs/books/tutorial/java/javaOO/accesscontrol.html>
(Accessed 13 March 2008).

For instance, from Figure 3, we know that the attribute *number* of class *Room* is of type *integer*; the figure does not represent visibility, we assume it to be the default visibility. This can be declared in Java as follows:

```
public class Room {
    // object attribute as an instance variable
    int number;
    //...
}
```

For a private attribute, the Java keyword `private` would be used. For instance, the attribute *creditCardDetails* of class *ReserverPayer* would be declared in Java as:

```
public class ReserverPayer extends Guest {
    // private variable
    private CreditCard creditCardDetails;
}
```

An example of the use of a class variable is in class *Reservation*. If we assume that the attribute *number* contains the total number of reservations made so far, any new reservation would have as its *number* the next higher value. Doing so would mean that a reservation's unique number corresponds to the order of the reservation in relation to all reservations made. For example, if there are 350 reservations made so far, the class variable would hold the value 350, and when the next reservation is made it would be allocated the number 351, and the class variable would be updated to 351. In Java, class variables are declared with the following syntax.

```
visibilityKeyword static variableType variableName;
```

Initialisation can also optionally be done in the same statement as the declaration.

```
private static int numberOfReservations = 0;
```

The value of the class variable can be accessed, from the class or any of its instances, using just the variable name *numberOfReservations*; there is no need to use the qualified name which is formed from the class name followed by the variable name, as in *Reservation.numberOfReservations*.

User-defined types

Programming languages generally come equipped with a wide selection of **predefined types**, which are readily available to the developer. These include, for instance, types for numbers, strings and collections. On the other hand, in developing a system, new types which are relevant only to the particular application being developed may need to be defined. In the hotel system, for instance, there are types like *Name*, *Address* and *CreditCard*. Such types will have to be represented in the target language either using the language's predefined types, or when these are not sufficient, as **user-defined types**. In developing a new user-defined type, you should take the following factors into consideration:

- ▶ the permissible set of values, which may well have been addressed during analysis, perhaps through the expression of class invariants;
- ▶ the availability of types within the class library of the programming language that can contribute to the definition of the new types;
- ▶ the opportunity for reuse of the new type across the system under development or across different systems.

We will look at a few examples. In the hotel system, we defined a type *RoomKind* in class *RoomType*. We never defined what this type was, but let us assume that *RoomKind* is an enumeration of values: *single*, *double*, *twin*, *familyOf3*, *familyOf4*, *familyOf5*. An **enumeration type** is a fixed set of values representing the various possible choices; a variable of an enumeration type is a reference to one of those values. From version 1.5 onwards, Java has supported enumerations and allowed new enumeration types to be defined by using the keyword `enum`. An enum type has no instances other than those defined by its enum constants; by convention, constants are written in upper case.

RoomKind could be declared as a new enumeration type in Java as follows:

```
// user-defined enumeration type
enum RoomKind {
    SINGLE, DOUBLE, TWIN, FAMILYOF3, FAMILYOF4, FAMILYOF5
}
```

This new type can then be used to declare variables, just like any other type, by using an ordinary Java variable declaration.



Exercise 1

Declare a public instance variable of type *RoomKind* in Java.

Solution

A possible solution is:

```
// variable of type RoomKind
RoomKind kind;
```

To access each of the *RoomKind* values we must use dot notation: *RoomKind.DOUBLE* is the instance *DOUBLE* of the type *RoomKind*, and so on.

Often introducing a user-defined type means defining a new class, which can then be added to the language library. Let us consider another example from the hotel system. During analysis we identified a type *Money*, which was used, for instance, in the definition of the *cost* attribute in *RoomType*. We could just use a predefined numerical type to represent *Money*. However, an amount of money is not just a number: for instance, it cannot be negative and it has an associated currency. Fowler (1997) discusses a pattern, called the **Quantity pattern**, which provides useful guidance on how to represent quantities such as amounts of money. Here is the pattern.

- ▶ *Name* Quantity.
- ▶ *Intent* To represent dimensioned quantities; for example: 6 ft, 30 kg.
- ▶ *How it works* The basic idea is to define a *Quantity* class that combines the amount with the unit.
- ▶ *When to use it* Every time you need to represent a quantity with both a value and a unit.
- ▶ *Examples* The height of a person, the weight of a package.

More on the Quantity pattern can be found at <http://www.martinfowler.com/ap2/quantity.html> (Accessed 22 February 2008).

Exercise 2

Explain a possible application of the Quantity pattern in the definition of the user-defined type *Money*.



Solution.....

Money can be seen as a quantity with a value (the amount) and a unit (the currency). Hence, *Money* can be represented as the class shown in Figure 4, where we have assumed that both attributes are private, with *amount* of type *Integer*, and *currency* of type *Currency* (an enumeration of, say, £, \$ and €).

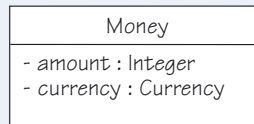


Figure 4 The user-defined type *Money*

Exercise 3

Give a Java definition of *Money* as described in Exercise 2. Note that `java.util` defines a class `Currency`.



Solution.....

```
public class Money {
    BigInteger amount;
    Currency currency;
}
```

We followed Fowler's implementation using `BigInteger` (used for high precision calculations).

We can also associate some other operations with *Money*; for instance, it is possible to add two amounts of the same currency, using a *plus* operation, or to calculate some multiple of the amount, using a *multiply* operation. The two operations are defined by the following contracts.

context *Money::plus*(someMoney : *Money*) : *Money*

pre:

-- the currency of *someMoney* and this *Money* object are the same

post:

-- the result of adding *someMoney* to this *Money* object will be

-- the *Money* object whose *currency* is the same as this *Money* object's *currency*

-- and whose *amount* is the sum of this *Money* object's *amount*

-- and *someMoney*'s *amount*

context *Money::multiply*(anAmount: *Integer*) : *Money*

pre:

-- none

post:

-- the result of multiplying this *Money* object by *anAmount* is the *Money* object

-- whose *currency* is the same as this *Money* object's *currency* and

-- whose *amount* is the product of *anAmount* and this money object's *amount*

An instance of *Money* can be created by invoking the following operation.

context *Money::create*(anAmount : *Integer*, aCurrency : *Currency*) : *Money*

pre:

-- none

post:

-- the result is a new *Money* object whose *amount* is *anAmount*

-- and whose *currency* is *aCurrency*

The type *Money*, including its operations, is shown in Figure 5.

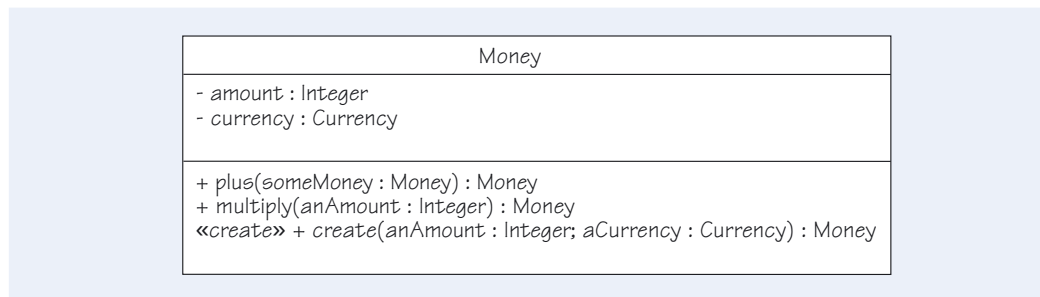


Figure 5 The user-defined type *Money*, showing its operations

The *plus* operation can be implemented in Java as the method `plus` below. Note that the `return` statement in the method body invokes the operation (constructor) which creates and returns an instance of *Money*. This implementation assumes the precondition has been checked.

```
/**
 * Returns a Money object whose currency is the same as
 * this object's currency and whose amount is the sum of this
 * object's amount and someMoney's amount.
 */
public Money plus(Money someMoney) {
    return new Money(this.amount.add(someMoney.amount), this.currency);
}
```

Exercise 4

Give a Java implementation of the operation *multiply* in type *Money*.



Solution.....

```
/**
 * Returns a Money object whose currency is the same as
 * this object's currency and whose amount is the
 * product of amount and this object's amount.
 */
public Money multiply(BigInteger amount) {
    return new Money(this.amount.multiply(amount), this.currency);
}
```

Implementing associations

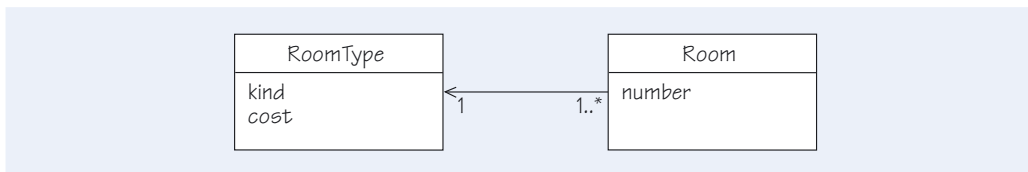
In *Unit 6* we discussed how objects collaborate by sending messages to each other. For an object to be able to send a message to another object, it needs to know the identity of the other object; that is, it must have, or be able to obtain, a reference to the other object. In a class diagram, an association that can be navigated from class *A* to class *B* tells us that objects of class *A* are able to send messages to objects of class *B*, and hence must know their identities. Also, the multiplicity of the association will tell us how many objects of class *B* are linked with a single object of class *A*. More precisely:

- ▶ if the multiplicity at the *B* end of the association is 'one', then each *A* object will hold a reference to exactly one *B* object;
- ▶ if the multiplicity at the *B* end of the association is 'many', then each *A* object will hold a number of references to *B* objects.

We look now at how to implement these associations in Java.

Association end with multiplicity 'one'

Consider the association from *Room* to *RoomType* in Figure 6. This association tells us that each *Room* object must hold the object identity of a *RoomType* object (the room



type of that room).

Figure 6 Association end with multiplicity 'one'

This can be implemented by defining a variable *roomType* of type *RoomType* in class *Room*: this will hold a reference to a *RoomType* object. The effect of this definition is illustrated in the object diagram in Figure 7.

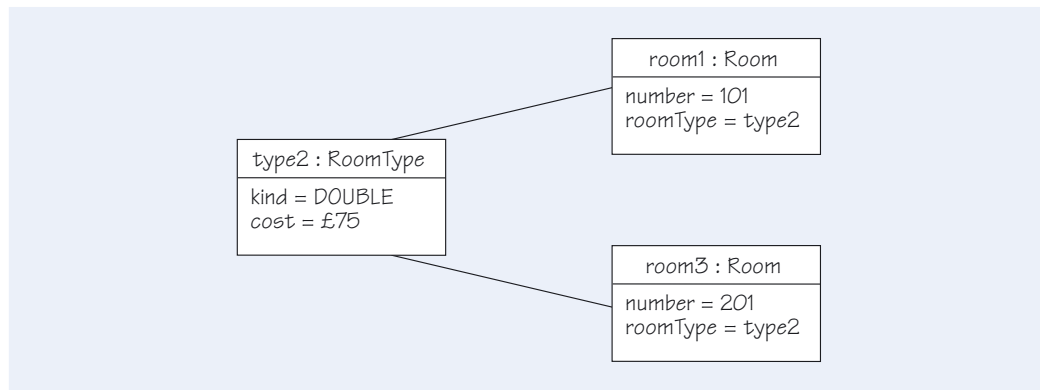


Figure 7 Object diagram with two *Rooms* and one *RoomType*

In Figure 7, the instance of *RoomType*, identified by the object reference *type2*, is referenced by both *Room* objects: their *roomType* variables hold such a reference.



Exercise 5

Give the Java declaration of *roomType* in class *Room*, as a private instance variable.

Solution

The Java declaration for *roomType* in class *Room* would be (assuming we have defined *RoomType*):

```
// association end with multiplicity 'one'
RoomType roomType;
```

Association end with multiplicity 'many'

We now consider the case where the association end has multiplicity 'many'.

SAQ 1

Consider the association from *Hotel* to *RoomType* in Figure 3. What do you know about objects linked by the association *Hotel* to *RoomType*?

ANSWER

The association from *Hotel* to *RoomType* expresses the fact that each *Hotel* object has to be able to hold the object identities of many *RoomType* objects.

Other associations with multiplicity 'many' have been qualified, to indicate that for each value of a qualifier attribute there is at most one object at the other end; however, they are still associations between one object at one end and many at the other end.

SAQ 2

What does Figure 3 tell us about the objects linked by the association *ReserverPayer* to *Reservation*?

ANSWER

The association from *ReserverPayer* to *Reservation* expresses the fact that each *ReserverPayer* object has to be able to hold the object identities of many *Reservation* objects, but for each *ReserverPayer* object and a reservation number there is at most one *Reservation* object.

In order to implement associations with multiplicity 'many' in a programming language, we need to use collection types: objects of these types group multiple objects into a single unit. In Java there are a number of similar classes implementing each type of collection. We will, therefore, focus not on particular classes but rather on the common interface that they implement. We look, in particular, at the following interfaces:

Set common behaviour for a collection of unordered elements with no duplicates;

List common behaviour for a collection whose elements are ordered and can be accessed by a numeric index that indicates their positions in the collection;

Map common behaviour for a collection in which the elements are pairs of key and value; a key maps to at most one value, but values can be duplicated.

We will use as implementation classes for these interfaces, respectively, the general purpose classes `HashSet`, `ArrayList` and `HashMap`.

In Java, collection types can be parameterised by the type of objects they contain: this type of collection is called a **generic collection**. For instance, to implement the *Room–Guest* association, a *Room* object will hold a reference to a collection object which will include the references to *Guest* objects for that room. We choose the `Set` interface here because we assume that there is no significant ordering on guests. We can declare an instance variable `guests` as a generic collection of interface type `Set` in class *Room* as follows:

```
// association end 'many' (using a generic collection)
Set<Guest> guests;
```

In the code, the angle brackets (< and >) indicate the type of the elements for this collection (*Guest* in this case). We could also declare the variable and assign a set to it all in one line:

```
Set<Guest> guests = new HashSet<Guest>();
```

This is illustrated in Figure 8, where the collection of items is a *HashSet* object, whose private instance variable, *value*, is the set of *Guest* object references.

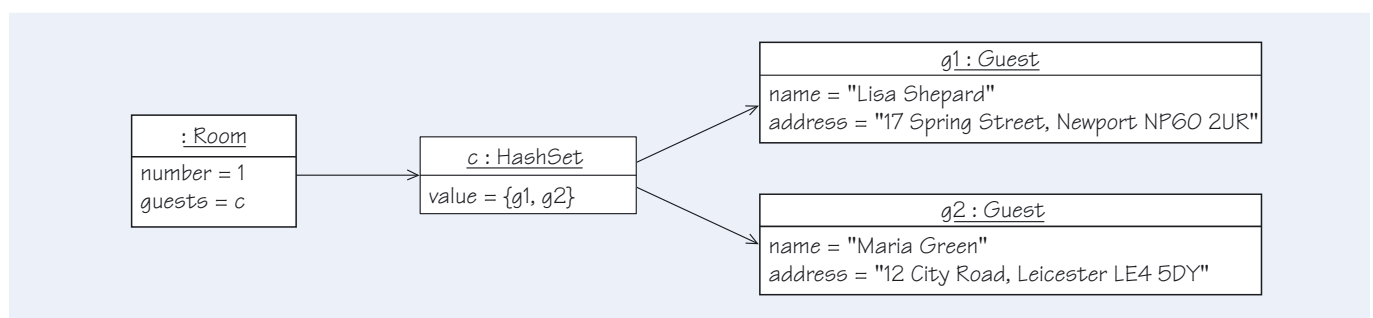


Figure 8 From one *Room* object to many *Guest* objects



Exercise 6

Consider the association shown in Figure 9.

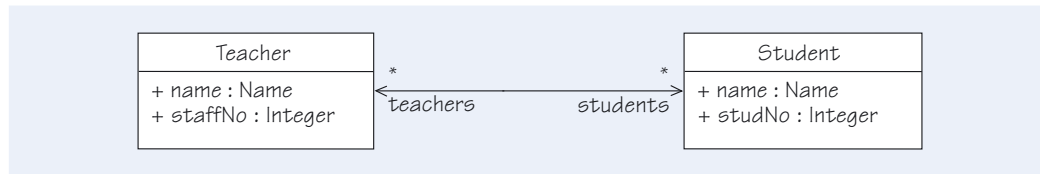


Figure 9 A teacher–student association

- What does it say about object references?
- How would you implement it in Java?
- Draw an object diagram to illustrate your implementation.

Solution

- Each *Teacher* object must hold the references to their *Student* objects; and each *Student* object must hold the references to their *Teacher* objects.
- We can declare a variable *teachers* in *Student* and a variable *students* in *Teacher*; both variables are declared as generic collections of interface type *Set* and can be assigned, respectively, *HashSet<Student>* to *teacher* and *HashSet<Teacher>* to *student*, as follows:

```

// in class Teacher
Set<Student> students = new HashSet<Student>();
// in class Student
Set<Teacher> teachers = new HashSet<Teacher>();
  
```

- An example of the implementation in part (b) is shown in Figure 10 below.

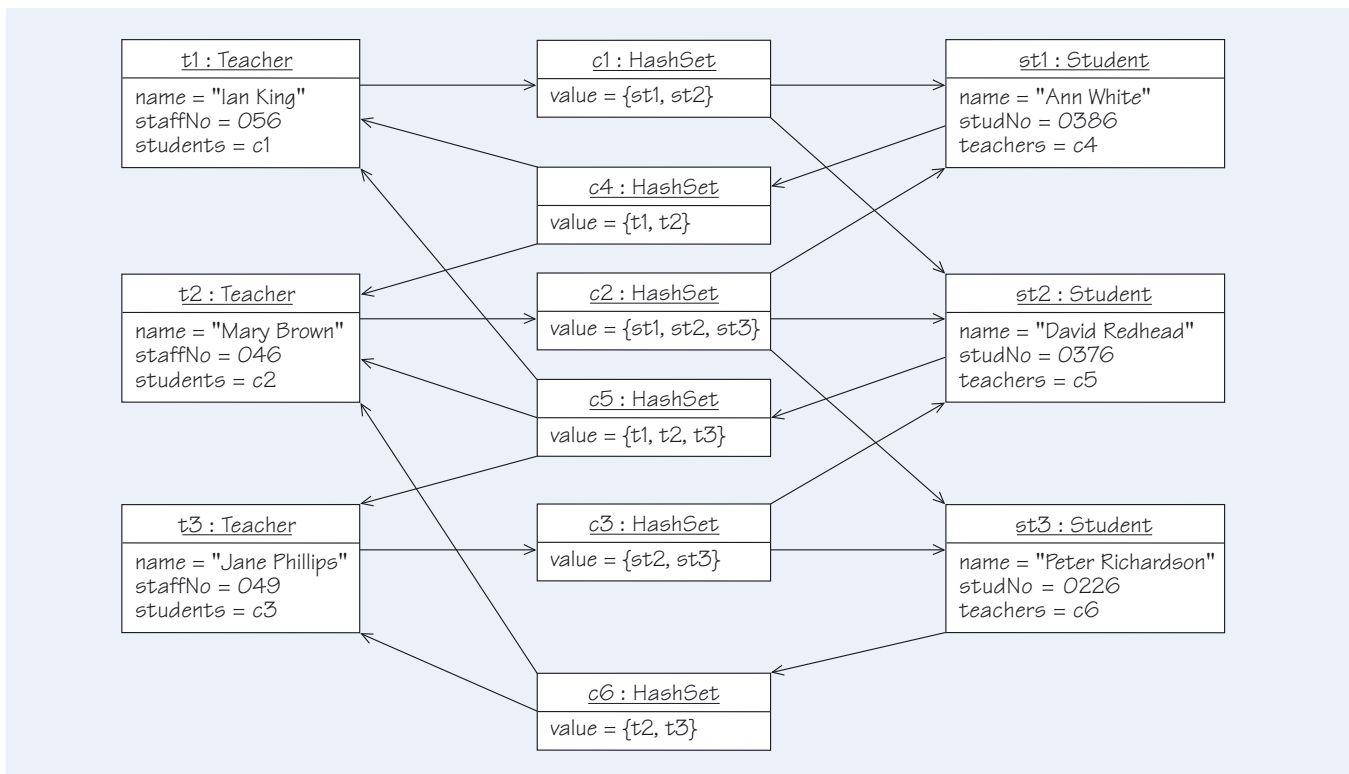


Figure 10 An object diagram showing the implementation of a many-to-many association

Exercise 7

Give a Java declaration to implement the association, shown in Figure 3, between *Hotel* and *RoomType*.



Solution

```
// in class Hotel
Set<RoomType> roomTypes;
```

Association classes

An association class imposes a restriction, in that there can be at most one instance of the association class per pair of objects of the associated classes.

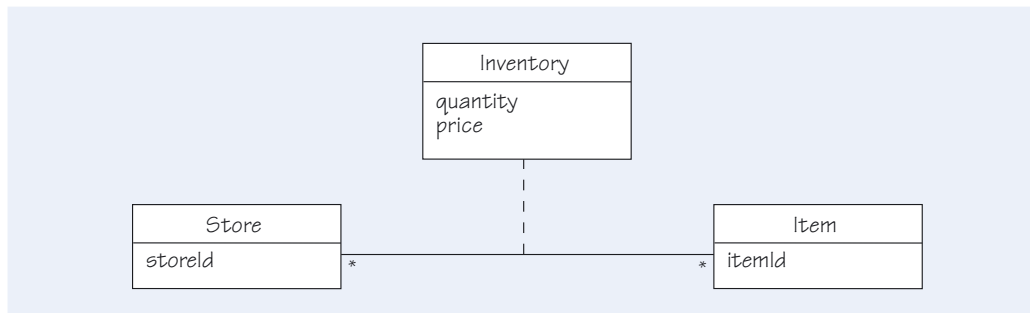


Figure 11 Association class

Figure 11 is part of a model for a chain of stores. Items can vary in price and quantity according to the store where they are stocked. The association class indicates that for each store and item there is exactly one value for the price and quantity of that item in that store.

An association class is implemented by a class with the association class attributes and the identifier attributes of each of the associated classes. In the example, a class *Inventory* would be declared with attributes *quantity*, *price*, *storeId* and *itemId*.

Exercise 8

Give the Java declaration of an association class to implement the association *HowMany* between *Reservation* and *RoomType*, in Figure 3.



Solution

We renamed the identifier attribute *number* of *Reservation* as *reservationNumber* to avoid conflict with *number* of *HowMany*.

```
public class HowMany {
    int number;
    int reservationNumber;
    RoomKind kind;
    ...
}
```

We leave out any further discussion of the use of association classes, for lack of space and as we will not be using them in the first iteration of the implementation.

Qualified associations

Consider the qualified association between *HotelChain* and *ReserverPayer* in Figure 3.

SAQ 3

What does this association tell us?

ANSWER.....

The association expresses the fact that *ReserverPayer* objects are identified in the system through their *id* (assigned to them the first time they register and used to represent their value identity). The *HotelChain* object is linked to all *ReserverPayer* objects via this association, and will perform a mapping between the value and object identities of *ReserverPayer* objects in order to be able to reference them.

Value and object identities were explained in *Unit 5*.

As explained earlier in this subsection, an association end with multiplicity 'many' can be implemented in a programming language by making use of collection types. In Java, the `Map` interface is particularly suited to representing the behaviour of qualified associations, because elements of a collection implementing the `Map` interface are key and value pairs, with keys being unique. By using value identities as keys and object identities as values, we have everything needed to represent a qualified association.

The following predefined methods of the `Map` interface are particularly relevant:

`put` expects a key and a value as parameters, and adds the pair to the collection;

`get` expects a key as a parameter, and returns the corresponding value, if found; otherwise returns `null` (the Java keyword for an undefined object reference).

The most generally useful class implementing the `Map` interface is `HashMap`. We can now represent the qualified association between *HotelChain* and *ReserverPayer* through the following declaration in class *HotelChain*:

```
Map<String, ReserverPayer> reservers;
```



Exercise 9

Give Java declarations to implement the following qualified associations:

- ▶ between *HotelChain* and *Hotel*;
- ▶ between *Hotel* and *Reservation*;
- ▶ between *Hotel* and *Room*;
- ▶ between *ReserverPayer* and *Reservation*.

Solution

- ▶ `Map<String, Hotel> hotels;` declared within the *HotelChain* class;
- ▶ `Map<Integer, Reservation> reservations;` declared within the *Hotel* class;
- ▶ `Map<Integer, Room> rooms;` declared within the *Hotel* class;
- ▶ `Map<Integer, Reservation> myReservations;` declared within the *ReserverPayer* class;

We have dealt so far with classes, attributes and associations; we will look at operations next.

You should now look at the code for the Hotel System. Look, in particular, at the different classes, and the declarations of attributes and associations, especially in *Hotel*, *Reservation*, *ReserverPayer*, *Room* and *Guest*.

3.2 Implementing operations

Operations are implemented as methods in an object-oriented programming language. Methods can be either instance or class methods (similar to instance and class variables). More precisely:

- ▶ **instance methods** correspond to object operations, that is, operations invoked on objects;
- ▶ **class methods** correspond to class operations, that is, operations invoked on classes.

Constructors

An important category of class operations are those used to create new class instances. They are referred to as constructors. By default, every class declared in Java has a constructor, used to create new instances, even if it is not declared explicitly. Constructors are invoked using the `new` keyword.

To declare a constructor explicitly in Java, the following syntax is used:

```
public className(parameterType parameterName, ...);
```

The constructor is named after the class and does not have an explicit return type. For instance, the constructor for the *Room* class can be expressed in Java as:

```
/**
 * Creates a new instance of Room with specified room number and type
 *
 * @param number    the room number
 * @param roomType  the room type
 */
public Room(int number, RoomType roomType) {
    this.number = number;
    this.roomType = roomType;
    guests = new HashSet<Guest>();
}
```

You will see later in the unit the need for the `roomReservations` instance variable (shown in the code).

Java has special syntax for invoking a constructor: the keyword `new` is used, as shown below.

```
new Room(number, roomType);
```

Exercise 10

Write the constructor for class `Reservation`.

Note: we use the abstract class `Calendar` defined in `java.util`; it converts between a `Date` object and a set of integer fields such as `YEAR`, `MONTH`, `DAY`, `HOURL` etc. A `Date` object represents a specific instant in time with millisecond precision. `getInstance(TimeZone zone)` and `getTime()` are methods of class `Calendar` that create a calendar using the specified time zone, and get the calendar's current time, respectively.



Solution

```
/**
 * Creates a new instance of Reservation with attributes initialised
 *
 * @param startDate    the start Date
 * @param endDate      the end Date
 * @param roomType     the RoomType
 * @param reserverPayer the ReserverPayer
 */
public Reservation(Date startDate, Date endDate,
                   RoomType roomType, ReserverPayer reserverPayer) {
    // increment class variable numberOfReservations
    // and use it to initialise number
    numberOfReservations++;
    number = numberOfReservations;
    // initialise other variables
    this.startDate = startDate;
    this.endDate = endDate;
    this.roomType = roomType;
    // set reservation date to current time and date
    reservationDate =
        Calendar.getInstance(TimeZone.getDefault()).getTime();
    // set the reserverPayer
    this.reserverPayer = reserverPayer;
}
```



Exercise 11

Write the constructor for class `Hotel`.

Solution

```
/**
 * Creates a new instance of Hotel
 *
 * @param name  the name of the new hotel
 */
public Hotel(String name) {
    this.name = name;
    reservations = new HashMap<Integer, Reservation>();
    rooms = new HashMap<Integer, Room>();
    roomTypes = new HashSet<RoomType>();
}
```

From operation contracts to methods

In order to express an operation in the design model as a method, we need to match the operation's signature to the method declaration and provide code in the method's body which realises the operation contract. For example, the operation *createReserverPayer* in class *HotelChain* was defined in *Unit 8*:

context `HotelChain::createReserverPayer(id : Id, name : Name, address : Address, creditCardDetails : CreditCard) : ReserverPayer`

A corresponding method could be defined in Java as indicated below, where the body of the method contains Java statements that realise the operation contract. Note that this operation invokes the constructor of `ReserverPayer` in order to create the new instance.

In class `HotelChain`:

```
/**
 * Creates a ReserverPayer with id, name, address and creditCardDetails
 * The ReserverPayer is added to the HotelChain's reservers
 * The ReserverPayer is returned
 *
 * @param reserverId      the unique id of the reserver
 * @param reserverName    the name of the reserver
 * @param reserverAddress the address of the reserver
 * @param reserverCreditCard the credit card used to make the reservation
 *
 * @return                the new ReserverPayer object
 */
public ReserverPayer createReserverPayer(String reserverId,
    String reserverName, String[] reserverAddress,
    CreditCard reserverCreditCard) {
    ReserverPayer reserverPayer;
    // invoke constructor, create link and return the new instance
    reserverPayer = new ReserverPayer(reserverId, reserverName,
        reserverAddress, reserverCreditCard);
    reservers.put(reserverId, reserverPayer);
    return reserverPayer;
}
```

Exercise 12

Write the Java method for the following operation.

context `Room::createGuest(name : Name, address : Address) : Guest`

Solution

In class `Room`:

```
/**
 * Create a guest and add this to the room
 *
 * @param guestName    the name of the guest
 * @param guestAddress the address of the guest
 *
 * @return             the new Guest object
 */
public Guest createGuest(String guestName, String[] guestAddress) {
    Guest guest;
    // invoke constructor, create link and return the new instance
    guest = new Guest(guestName, guestAddress);
    this.guests.add(guest);
    return guest;
}
```



3.3 More complex operations

So far we have dealt mostly with the basic structure of the design class diagram. We look now at some of the more complex operations required for the implementation of the use cases being addressed. In this section we show the implementation of *make reservation* and *cancel reservation*. We start with *make reservation*.

context HotelChain::makeReservation(hotelName : Name, startDate : Date, endDate : Date, roomType : RoomType, name : Name, address : Address, creditCardDetails : CreditCard) : String

pre:

- *startDate* is before *endDate*
- there is a *Hotel* with *Name* *hotelName*, and it has *RoomTypes* of *roomType*

post:

- if *available(startDate, endDate, roomType)* executed in the *Hotel* with *hotelName*
- returned true then
- a new *Reservation* object, *reservation*, will have been created (object creation)
- if there is not a *ReserverPayer* with given details in that *Hotel* then
- a new *ReserverPayer* object will be created with attributes set to
- *name*, *address*, *creditCardDetails* and a newly generated *id*
- the new *ReserverPayer* object will be linked to *hotelChain*
- that *Hotel* will be linked to *reservation*
- that *ReserverPayer* will be linked to *reservation*
- *reservation* will be linked to *roomType*
- *reservation* will be linked to a previously free *Room*
- a string indicating the reservation *number* will have been returned
- otherwise, a string indicating failure will have been returned

Verifying the precondition of make reservation

In *Unit 8* we separated the verification of the precondition from the fulfilment of the postconditions. The former was specified as follows.

context HotelChain::canMakeReservation(hotelName : Name, startDate : Date, endDate : Date, roomType : RoomType) : Boolean

post:

- true will be returned if
- *startDate* is before *endDate*, and
- there is a *hotel* with *Name* *hotelName*, and it has rooms of *roomType*
- otherwise, false will be returned

In the implementation, it is the responsibility of the user interface to check the preconditions, as you will see in Section 5; therefore we will not discuss the implementation of this operation here. We could have used assertions, as discussed in *Unit 9*, to help with debugging a program and providing assurance that the program is behaving as intended. We have, however, decided not to use them in the first iteration of developing the code, to keep the process simple.

Fulfilling the postcondition of make reservation

In *Unit 8* we allocated the responsibilities to fulfil the postcondition as shown in Figure 12.

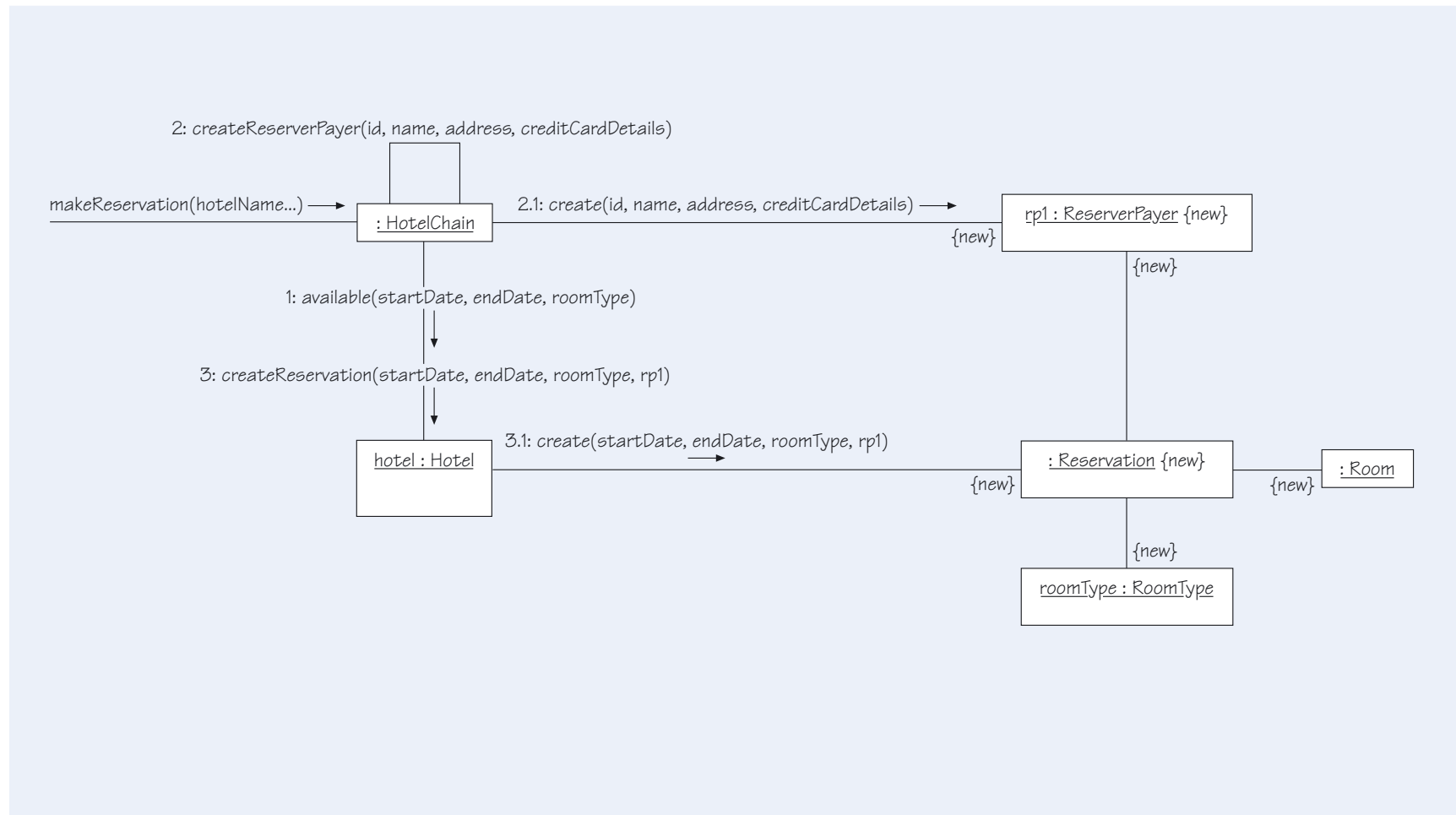


Figure 12 Allocation of responsibilities in design

We now need to define an operation in *Hotel* to check the availability of rooms of a given room type for specific dates, and a second operation in *Hotel* to create a reservation.

Checking availability

context *Hotel::available*(startDate : Date, endDate : Date, roomType : RoomType) : Boolean

post:

-- true will have been returned if there is a room of the required room type that
 -- has no reservations that overlap with the period between *startDate* and
 -- *endDate* inclusive

This indicates that we need to know which reservations there are for a room. In *Unit 8* we saw that the association between *Room* and *Reservation* was navigated in the direction from *Reservation* to *Room*. The specification above shows that this association also needs to be navigable from *Room* to *Reservation*.

SAQ 4

How do you express this new navigability in Java?

ANSWER.....

The following instance variable declaration would need to be added to class *Room*.

```
Set<Reservation> roomReservations;
```

Because a *Room* needs to know about all *Reservations*, we have opted to use a *Set* rather than a *Map*. Had this been a qualified association whereby a specific reservation could be identified from its number, this would no longer be the case.

The *Room* constructor would be modified as follows.

```
/**
 * Creates a new instance of Room with specified room number and type
 *
 * @param number    the room number
 * @param roomType  the room type
 */
public Room(int number, RoomType roomType) {
    this.number = number;
    this.roomType = roomType;
    guests = new HashSet<Guest>();
    roomReservations = new HashSet<Reservation>();
}
```

The postcondition of the *available* method is quite complex. What does it mean to overlap with the period between *startDate* and *endDate*? An existing reservation *res* will overlap if *res.startDate* is before *endDate* and *res.endDate* is after *startDate*.

To write the *available* operation in Java we need to remind ourselves about the use of iterators in Java. We need to iterate through the set of rooms of a hotel, and for each room we need to check whether there is a reservation that clashes with the required dates. We will do this check with a separate method *checkAvailability*, defined below.

```
/**
 * Check that the proposed reservation dates for the specified
 * room type have at least one room with no clashing
 * reservation dates.
 *
 * @param startDate    proposed reservation start date
 * @param endDate      proposed reservation end date
 * @param roomType      the room type required for the reservation
 *
 * @return             whether at least one room of the required type
 *                     is available on the specified dates
 */
public boolean available(Date startDate, Date endDate,
                        RoomType roomType) {
    Collection<Room> allRooms = rooms.values();
    Iterator<Room> roomIterator = allRooms.iterator();
    boolean available = false;
    // an explicit iterator is used instead of foreach
    // so we can return as soon as a room tests available
    while (!available && (roomIterator.hasNext())) {
        Room currentRoom = roomIterator.next();
        if (currentRoom.isRoomType(roomType)) {
            available = checkAvailability(startDate, endDate, currentRoom);
        }
    }
    return available;
}

private boolean checkAvailability(Date startDate, Date endDate,
                                Room room) {
    Collection<Reservation> allReservations = room.getRoomReservations();
    boolean available = true;
    for (Reservation eachReservation : allReservations) {
        // The current reservation will clash when it starts before
        // the end date we are interested in or finishes after
        // the start date we are interested in
        if ((eachReservation.startDate.before(endDate)) &&
            (eachReservation.endDate.after(startDate))) {
            available = false;
        }
    }
    return available;
}
```

This syntax for the *for loop*, called the *enhanced for loop*, is specifically designed for iteration over collections and arrays.

Creating a reservation

We can now think about the operation in *Hotel* that creates the reservation and establishes appropriate links to other objects.

context `Hotel::createReservation(startDate : Date, endDate : Date, roomType : RoomType, reserverPayer : ReserverPayer) : Reservation`

pre:

-- none

post:

-- a new instance of *Reservation* will have been created and initialised with the

-- given values

-- the new instance will have been added to the collection of reservations for

-- *reserverPayer*

-- the new instance will have been added to the collection of reservations for the *Hotel*

-- a *Room* will have been allocated for the new instance

-- the new instance will have been returned

This operation invokes the constructor in *Reservation* to obtain a new reservation, links reservation and the *Hotel*, reservation and *reserverPayer*, reservation and a *RoomType*, and reservation and a *Room*. We will define the following ancillary operations:

- ▶ *addReservation(Reservation)* in *Hotel* to establish the link with the new reservation;
- ▶ *addReservation(Reservation)* in *ReserverPayer* to establish the link with the new reservation;
- ▶ *availableRooms(Date, Date, RoomType)* in *Hotel* to obtain all possible *Rooms*;
- ▶ the operation *allocatedTo(Reservation)* in *Room*, and the setter *setAllocatedRoom(Room)* in *Reservation*, to establish the bidirectional link between the new reservation and its allocated *Room*.

We show here the code for the *addReservation* operations in both *Hotel* and *ReserverPayer*.

In class *Hotel*:

```
/**
 * Add the reservation to the hotel's record of reservations (HashMap)
 *
 * @param reservation the reservation to add to the records
 */
void addReservation(Reservation reservation) {
    reservations.put(reservation.getNumber(), reservation);
}
```

In class *ReserverPayer*:

```
/**
 * Add a reservation to the reservers list (HashMap) of reservations.
 *
 * @param reservation the reservation object
 */
public void addReservation(Reservation reservation) {
    myReservations.put(reservation.getNumber(), reservation);
}
```

Exercise 13



Write on paper the Java code for `setAllocatedRoom(Room)` and `allocatedTo(Reservation)` defined as:

context `Reservation::setAllocatedRoom(aRoom : Room)`

context `Room::allocatedTo(aReservation : Reservation)`

Solution

```
/**
 * Set the allocated room to this reservation
 *
 * @param room the room to allocate to this reservation
 */
public void setAllocatedRoom(Room room) {
    allocatedRoom = room;
}

/**
 * Allocates the room by adding a reservation
 *
 * @param reservation the reservation to allocate the room to
 */
public void allocatedTo(Reservation reservation) {
    roomReservations.add(reservation);
}
```

The operation that finds the available rooms is specified as follows.

context `Hotel::availableRooms(startDate : Date, endDate : Date, roomType : RoomType) : Set(Room)`

post:

- all the rooms of the required room type that have no reservations
- that overlap with the period between *startDate* and *endDate*
- will have been returned

In the implementation of this operation we should make it return a list of rooms, so that getting an available room is simply done by getting the first room from that list.



Exercise 14

Write on paper the Java code for `availableRooms` as specified above. Use for guidance the code written for the operation `available`, noting the differences in returned type and how to construct the returned value.

Solution

```
/**
 * Return a list of available rooms that fit the proposed reservation
 * dates for the specified room type
 *
 * @param startDate proposed reservation start date
 * @param endDate   proposed reservation end date
 * @param roomType  the room type required for the reservation
 *
 * @return          the set of rooms of the required type
 *                  available between the specified dates
 */
public List<Room> availableRooms(Date startDate, Date endDate,
                                RoomType roomType) {
    // initialise an empty set of available rooms
    List<Room> allAvailableRooms = new ArrayList<Room>();
    // get the collection of all rooms of hotel
    Collection<Room> allRooms = rooms.values();
    // create an iterator to search through all rooms
    Iterator<Room> roomSearcher = allRooms.iterator();
    // search all rooms while still rooms to search
    while (roomSearcher.hasNext()) {
        // set next room as current
        Room currentRoom = roomSearcher.next();
        // initially room is not known to be available
        boolean availableRoom = false;
        // only proceed if room is of correct type
        if (currentRoom.isRoomType(roomType)) {
            // get all the reservations recorded for this room
            Collection<Reservation> allRoomReservations =
                currentRoom.getRoomReservations();
            // create iterator to search all reservations for a room
            Iterator<Reservation> reservationSearcher =
                allRoomReservations.iterator();

            // presume available till proven otherwise
            availableRoom = checkAvailability(startDate, endDate, currentRoom);
        }
        // if room is available
        if (availableRoom) {
            // add to set of available rooms
            allAvailableRooms.add(currentRoom);
        }
    }
    // return the set of available rooms
    return allAvailableRooms;
}
```

We can now complete the implementation of *createReservation*. Note that this operation is only called if the precondition of *makeReservation* is true, and so we can assume that there are available rooms.

context *Hotel::createReservation(startDate : Date, endDate : Date, kind : RoomType, reserverPayer : ReserverPayer) : Reservation*

```
/**
 * Creates a reservation for this hotel for the specified dates,
 * room type and customer.
 * This should not be called unless there are rooms available of
 * the right type between the specified dates.
 * The reservation is added to the reservation records.
 * The reservation is added to the customer's records.
 * The room is allocated to the reservation.
 *
 * @param startDate      the start date for the reservation
 * @param endDate        the end date for the reservation
 * @param roomType        the type of room required (double/single/twin etc.)
 * @param reserverPayer  the customer who is reserving the room
 *                       and who will pay later
 *
 * @return               the new reservation object
 */
public Reservation createReservation(Date startDate, Date endDate,
    RoomType roomType, ReserverPayer reserverPayer) {
    // create a new reservation object
    Reservation newReservation =
        new Reservation(startDate, endDate, roomType, reserverPayer);
    // add the reservation to the hotel's reservations
    addReservation(newReservation);
    // add the reservation to the reserverPayer's reservations
    reserverPayer.addReservation(newReservation);
    // there is at least one available room
    List<Room> possibleRooms = availableRooms(startDate,
                                                endDate, roomType);

    //get first available room
    Room chosenRoom = (Room)possibleRooms.get(0);
    // allocate the room for the reservation
    chosenRoom.allocatedTo(newReservation);
    newReservation.setAllocatedRoom(chosenRoom);
    // return the created reservation
    return newReservation;
}
```

Implementation of makeReservation

We can now complete the implementation of the *makeReservation* operation. The specification of this operation needs to be slightly modified as follows: instead of a credit card object it takes a credit card number as a parameter and creates a credit card (with only name, address, and number) if the reserver is new. This is a reasonable simplification for this iteration.

context HotelChain::makeReservation(hotelName : Name, startDate : Date, endDate : Date, roomType : RoomType, String name, String[] address, String cardNumber) : String

pre:

- *startDate* is before *endDate*
- *roomType* is one of *hotel*'s *roomTypes*

post:

- if available(startDate, endDate, roomType) executed in *hotel* with *hotelName*
- returned true then
- a new *Reservation* object, *reservation*, will have been created (object creation)
- if there is not a *ReserverPayer* with given details in that *hotelChain* then
- a new *ReserverPayer* object will be created with attributes set to *name*, *address*,
- a newly created *creditCard* and a newly generated *id*
- the new *ReserverPayer* object will be linked to *hotelChain*
- that *hotel* will have been linked to *reservation*
- the *ReserverPayer* will have been linked to *reservation*
- *reservation* will have been linked to *roomType*
- *reservation* will have been linked to a previously free *room*
- a string indicating the reservation number will have been returned
- otherwise a string indicating failure will have been returned

The Java implementation is as follows. We follow the specification above, although you will realise in Section 5, that the check of availability of rooms becomes redundant here as this check will have been performed by the user interface before a reservation is made. For testing purposes, the code in the prototype has an additional parameter for today's date.


```
/**
 * Make a reservation with the details if able and return the results
 *
 * @param hotelName          the name of the hotel for the reservation
 * @param startDate          the start date for the reservation
 * @param endDate            the end date for the reservation
 * @param roomType           the type of room required
 * @param reserverName       the name of the reserver
 * @param reserverAddress    the address of the reserver
 * @param reserverCreditCardNumber the credit card number used by the
 *                             reserver to make the reservation
 *
 * @return                   a string indicating success or failure
 *                             and giving feedback
 */
public String makeReservation(String hotelName, Date startDate,
    Date endDate, RoomType roomType, String reserverName,
    String[] reserverAddress, String reserverCreditCardNumber) {
    Hotel hotel = getHotelByName(hotelName);
    ReserverPayer reserverPayer = null;
    Reservation reservation;
    String reservationMessage;

    // if available(...) in hotel returns true
    // a new reservation will be created
    if (hotel.available(startDate, endDate, roomType)) {
        // does a ReserverPayer with this name and address exist?
        reserverPayer = getExistingReserverPayer(reserverName, reserverAddress);
        // if ReserverPayer doesn't exist create a new one
        if (reserverPayer == null) {
            CreditCard creditCard = createCreditCard(reserverName,
                reserverAddress, reserverCreditCardNumber);
            reserverPayer = createReserverPayer(Integer.toString(reserverId++),
                reserverName, reserverAddress, creditCard);
        }
        // make reservation
        reservation = hotel.createReservation(startDate, endDate, roomType,
            reserverPayer);
        reservationMessage = "Confirmed. Reservation number: " +
            reservation.number;
    } else {
        reservationMessage = "No room available of the required type";
    }
    return reservationMessage;
}
```

Implementation of *cancelReservation*

In *Unit 8* the operation for the *cancelReservation* use case was defined as follows.

context `HotelChain::cancelReservation(hotelName : Name, reservationNumber : Number)`

pre:

-- the *Reservation* with *reservationNumber* is linked to the *Hotel* with *hotelName*
 -- the *startDate* for the *Reservation* with *reservationNumber* is not in the past

post:

-- the *Reservation* with *reservationNumber* will no longer be linked to the *Hotel*
 -- with *hotelName*, to the *reserverPayer* who made it, nor to the allocated *room*
 -- or *roomType*

The verification of preconditions is the responsibility of the user interface. We will now look at the code for the following ancillary operations:

- ▶ *removeReservation(int)* in *ReserverPayer* to remove the reservation from its *reserverPayer*'s reservations;
- ▶ *deallocateRoom(Reservation)* in *Room* to deallocate the room allocated to the reservation;
- ▶ *cancelReservation(int)* in *Hotel* to cancel the reservation with the specified number.

removeReservation(int) in *ReserverPayer*

```
/**
 * Removes a reservation with the specified number
 * from the reserver's list(HashMap) of reservations
 *
 * @param reservationNumber the unique id of the reservation to remove
 */
public void removeReservation(int reservationNumber) {
    // remove the reservation from the reservations HashMap
    myReservations.remove(new Integer(reservationNumber));
}
```

deallocateRoom(Reservation) in *Room*

```
/**
 * Deallocates the room by removing a reservation
 *
 * @param reservation the reservation to deallocate from the room
 */
public void deallocateRoom(Reservation reservation) {
    // remove the reservation allocation (deallocate)
    roomReservations.remove(reservation);
}
```

cancelReservation(int) in Hotel

```
/**
 * Cancels the specified reservation by removing the reservation
 * from the hotel's reservations, removing it from the reserverPayer's
 * reservations and deallocating the room allocated to this
 * reservation.
 *
 * @param reservationNumber the number of the reservation to cancel
 *
 * @return feedback text about the cancelled reservation
 */
public String cancelReservation(int reservationNumber) {
    // if reservation contained in the records
    if (reservations.containsKey(reservationNumber)) {
        Reservation reservation = reservations.get(reservationNumber);
        // get the allocated room for this reservation
        Room allocatedRoom = reservation.getAllocatedRoom();
        // can't cancel if someone has checked in with this reservation number
        if (allocatedRoom.getCheckInReservationNumber() == reservationNumber) {
            return("Can't cancel - guest is checked in with this reservation");
        } else {
            // remove it from the reservations of the hotel
            reservations.remove(reservationNumber);
            // remove it from the reserver/payer's reservations
            reservation.getReserverPayer().removeReservation(reservationNumber);
            // deallocate the allocated room for this reservation
            allocatedRoom.deallocateRoom(reservation);
        }
        // return feedback that the reservation has been cancelled
        // along with the reservation info
        return "Reservation CANCELLED: " + reservationNumber;
    } else {
        return("Reservation number " + reservationNumber +
            " not found in hotel " + this.name + " records");
    }
}
```



Exercise 15

Write the Java code for *cancelReservation* in *HotelChain* as specified above.

Solution

```
/**
 * Cancel the reservation with the specified reservation number
 * in the given hotel name
 *
 * @param hotelName      the name of the hotel to cancel the
 *                        reservation
 * @param reservationNumber the reservation number to cancel
 *
 * @return               feedback whether the cancellation
 *                        was successful
 */
public String cancelReservation(String hotelName, int reservationNumber) {
    Hotel hotel = getHotelByName(hotelName);
    return hotel.cancelReservation(reservationNumber);
}
```

3.4 Summary of section

This section dealt with the implementation of the model. We had as a starting point the design class diagram from *Unit 8*, and we discussed how each element of the model is implemented. We also looked at how to implement operations given their specification.

4 The Façade pattern

The Façade pattern provides a unified interface for a set of interfaces; we use it here to offer two main facilities:

- 1 operations corresponding to the use cases (for example, *makeReservation*) in the model;
- 2 operations that make available, to the user interface, additional information to be displayed.

Both sets of operations convert objects and other data between different representations preferred by the user interface and the model respectively; they provide ‘impedance matching’ between the user interface and the model. The user interface should know little about the model, and so it uses data types that are primitive data types, utility types like `Date` and `String` that are built into Java, or objects that it has limited ability to manipulate as they have been supplied as being of type `Object` with no further type information (we explain more about this below). The Façade pattern when applied to a Java program is responsible for bundling data supplied in variables of primitive types into domain objects, and coercing objects of type `Object` into types that are more meaningful to the model.

A key role of the Façade interface, then, is to take objects from the domain model and pass them to the user interface as objects of type `Object`. This limits the way in which the user interface can manipulate them, but does allow the user interface to obtain a string that can be displayed. When such objects are passed back to the model layer, the Façade converts them back to their original, more informative type, allowing them to be manipulated more meaningfully.

Figure 13 shows a user interface which is the view/controller, a Façade hiding the model, and the communication between the user interface and the Façade, and between the Façade and the model.

So that the coupling between the view and the model is kept to a minimum, the user interface deals only in:

- (a) primitive data types;
- (b) built-in Java data types such as `String` and `Date`;
- (c) objects of type `Object`, with no other type information (because such information would be implementation-dependent and so increase coupling).

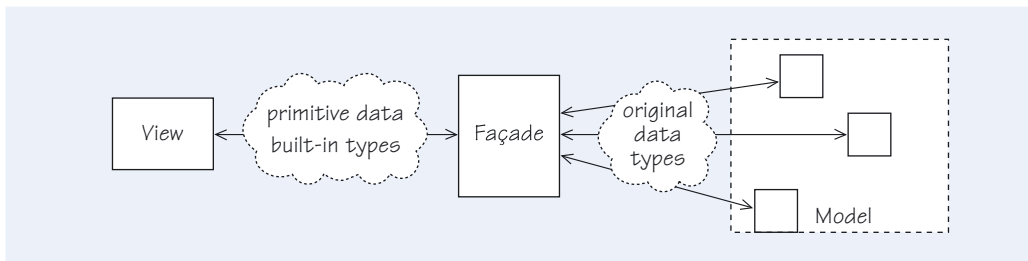


Figure 13 A model hidden from a user interface by a Façade

Since the view has no type information about items in category (c), it can only:

- ▶ invoke a very limited set of operations on them, such as `equals` and `toString` (but this is sufficient, for example, to populate a pick list in which objects will be displayed according to the result of invoking `toString` on them); or
- ▶ pass back references to objects the user selects into the model, where they can be cast back to their original type as required by the business methods.

You should now look at the code for the Façade.

We now illustrate the Façade pattern with two examples. For the hotel system, the user interface needs to display the hotels that belong to the hotel chain. The method *getHotels* of the Façade supports this:

```
/**
 * Return the array of hotels in the hotel chain
 *
 * @return the array of hotels in the hotel chain
 */
public Object[] getHotels() {
    return (Object[]) hotelChain.getHotelsArray();
}
```

For the reasons discussed above, it returns a list of objects of type `Object`, rather than of type `Hotel`. The `HotelChain` object records the hotels it knows about using a `HashMap` collection type. `HashMap` has a method `values` that will give us the `Hotel` objects as a `Set`; we then convert this to an array, as this form is more useful to the user interface.

```
/**
 * Return the array of hotels in the hotel chain
 *
 * @return the array of hotels in the hotel chain
 */
public Hotel[] getHotelsArray() {
    return (Hotel[]) getHotels().values().toArray();
}
```

Our second example shows how we support use cases. The method *makeReservation* of the Façade takes parameters supplying the data needed to carry out the use case. This method invokes the corresponding method in *HotelChain* but we do need to coerce the types of the *RoomType* object to make it more meaningful to the model.

```
public String makeReservation(String hotelName, Date startDate,
    Date endDate, Object roomType, String reserverName,
    String [] reserverAddress, String reserverCreditCardNumber) {
    return hotelChain.makeReservation(hotelName, startDate, endDate,
        (RoomType) roomType, reserverName, reserverAddress,
        reserverCreditCardNumber);
}
```

4.1 Summary of section

In this section we saw how the Façade pattern can be used to deal with the mismatch between the user interface and the model, at the same time maintaining low coupling between them.

5

The user interface

The application will be used for different purposes by different classes of users, who may also have different usability requirements, and we would probably need to provide each class of user with its own interface. However, the four use cases associated with the partition that we are considering all represent the functionality required by receptionists, and so we can adopt a relatively simple approach and use a single interface.

Our application will provide the ability to switch between the four use cases easily, and simply needs to allow the user to supply all of the parameters used by the respective system operations.

The user interface has been implemented making use of the user interface building tools in the NetBeans IDE. Figures 14–17 show how a tabbed panel has been used, each figure showing the panel for one of the four use cases. In each case the basic approach is that, on pressing a button, data is collected from some text fields (and a text area for the address associated with a reservation), and a call to an appropriate system operation is made. In the case of reservations we have two buttons. The first allows us to check whether a reservation is possible before going on to collect the reserver's details.

Swing's approach to user interfaces is based on a listener model: when buttons are pressed appropriate methods are invoked. We have kept the details fairly simple by just using this model to deal with events generated by the buttons.

Looking at the panels of the user interface, for example, for the *make reservation* use case, we can see that while they support the appropriate use case, they provide extra flexibility in that details do not have to be entered in a single fixed order. Furthermore, the *make reservation* use case, by providing a button to check availability of a room of the chosen room type for the dates requested, adds further convenience for the user. The lesson here is that constructing an interface for a system and therefore making its operation more tangible may suggest ways in which we can deliver benefits to users of the system that may not have been articulated in the requirements document. So our system, as well as delivering an increment of functionality, also has a role as a prototype, and can be used as a way of elaborating and refining requirements and even discovering new requirements. Indeed, a more agile approach to development might well carry out use case elicitation alongside user interface development. The user interface would be developed as a series of prototypes to illustrate the functionality required.

As we mentioned in Subsection 3.3, the checking of the preconditions for each of the use cases is the responsibility of the user interface. This is achieved in different ways: by listing only existing hotels (`hotelNameComboBox`) and offering within each hotel only room types that the hotel has (`roomTypeComboBox`); by checking that the values entered, for start and end dates for example, are correct dates, but also that the start date is before the end date. The former is shown in Subsection 5.4 and the latter in Subsection 5.1.

The screenshot shows a 'Hotel System' window with a 'Make Reservation' panel. The panel has four tabs: 'Make Reservation', 'Cancel Reservation', 'Check In', and 'Check Out'. The 'Make Reservation' tab is active. It contains the following elements:

- Hotel Name:** A dropdown menu with 'Druges' selected.
- Room Type:** A dropdown menu with 'DOUBLE' selected.
- Start Date (dd/mm/yy):** An empty text input field.
- End Date (dd/mm/yy):** An empty text input field.
- Check if room is available:** A button.
- Name:** A text input field.
- Address:** A text input field with a sub-button 'First address if previous customer'.
- Credit Card No (16 digits - spaces allowed):** A text input field.
- Confirm reservation:** A button.
- Reset form to make new reservation:** A button.
- Change today's date for testing (leave blank for system date):** A label and an empty text input field.

Note that the code is a prototype and while the ability to find duplicate names would be desirable it hasn't been implemented in this version of the code.

Figure 14 The 'Make Reservation' panel

Figure 14 shows the prototype user-interface design for the *make reservation* use case. You will have a fair idea of how it works from your experience of other user interfaces, but running and using the prototype will allow you get a better feel for how it corresponds to its intended use.

You should now look at the code for the user interface.

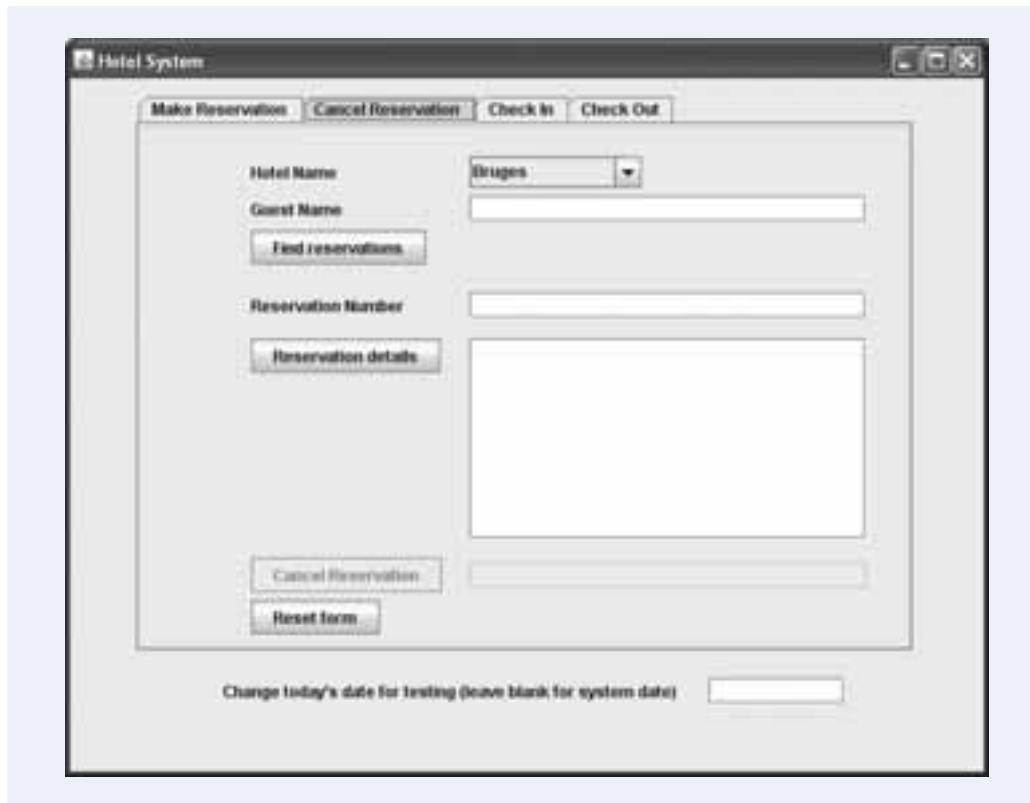


Figure 15 The 'Cancel Reservation' panel

Figure 15 shows the prototype user-interface design for the *cancel reservation* use case. To cancel a reservation the user must select the hotel name, and enter either the reservation number or the guest name to select from any reservations for that name. Although reservation numbers are unique across the system, asking for the hotel name introduces some redundancy that may be useful in reducing errors.

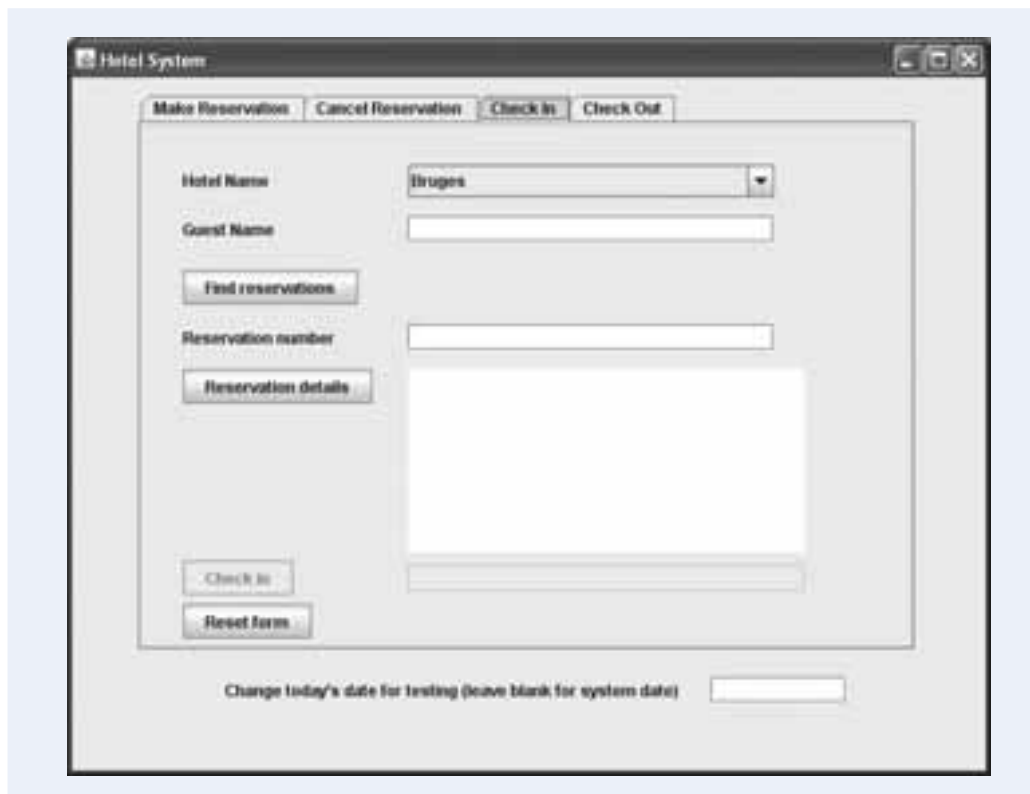


Figure 16 The 'Check In' panel

For simplicity, the implementation of the *check in* use case in the prototype is not consistent with the design in *Unit 8*.

Figure 16 shows the prototype user-interface design for the *check in guest* use case. To check in a guest, the user selects a hotel and enters the guest's name or reservation number. Arguments similar to those given previously apply in respect of redundancy.

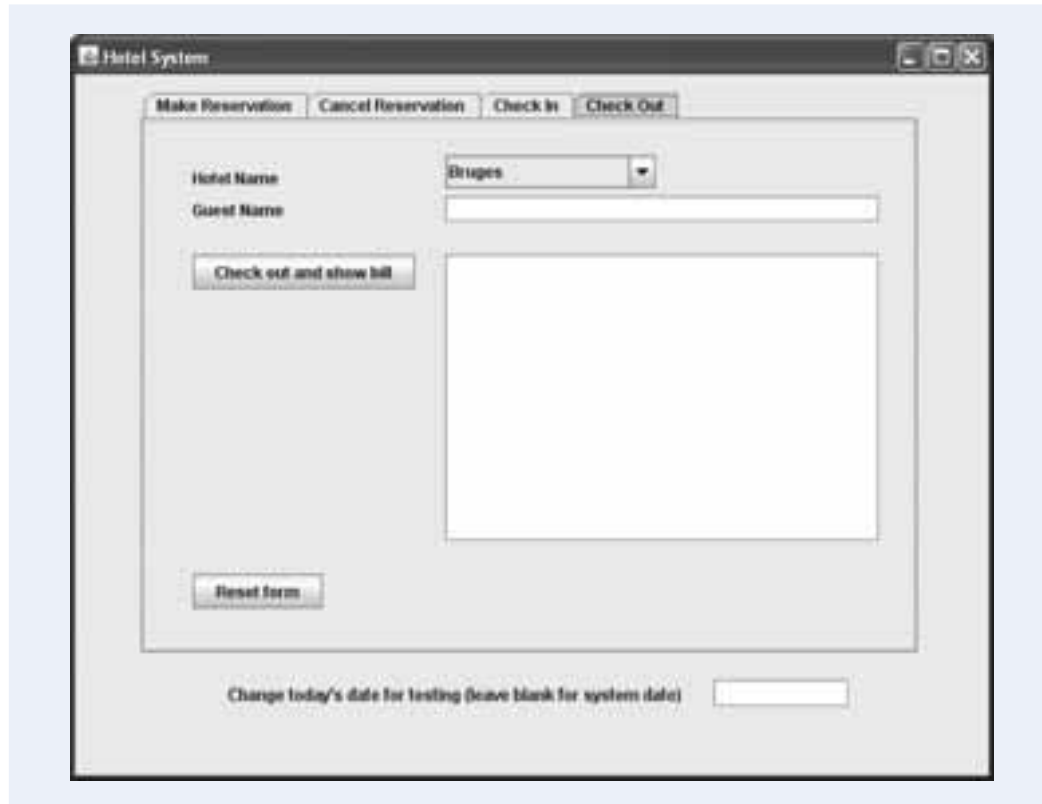


Figure 17 The 'Check Out' panel

Figure 17 shows the prototype user-interface design for the *check out guest* use case. The check out is done with the hotel and guest names. This prototype does not consider any further complexities.

In our case the main code providing the functionality behind the interface can be grouped as follows.

- 1 Some buttons require complex functionality – such as *Check if room is available*. The data required for this check (including hotel, room type and dates) is collected, and the check is performed (see Subsection 5.1).
- 2 Handling the main use case buttons. When one of the use case buttons – for example, the one labelled *Confirm reservation* – is pressed, we need to collect data from the appropriate text fields and pass them as parameters to the business layer via the Façade pattern (see Subsection 5.2).
- 3 We need to know when a name has been entered. We could do this in different ways: we could listen for the event that occurs when the user moves the focus of the interface from the *Name* field; or we could have a button that requests the addresses once the name has been entered – *Find address if previous customer*, or *Find reservations* to get the reservations for a name. We have chosen the second option (see Subsection 5.3).
- 4 When the user updates the hotel selected in the relevant combo-box, we need to ensure that the room types displayed are updated appropriately (see Subsection 5.4).
- 5 There needs to be some way to clear the fields on each panel. We have chosen to do this by having a reset button (see Subsection 5.5).

We now look briefly at some examples of the code associated with the above functionality. Bear in mind when looking at the source code that much of it is generated automatically, and therefore it is sensible to focus on the listener methods where the programmer supplies his or her own code to add functionality to the skeleton interface.

5.1 Handling the check of available rooms for the *make reservation* use case

The following instance variables are declared in the class definition. They are needed to collect and display information, and to refer to the Façade.

```
public class HotelUI extends javax.swing.JFrame {
    HotelFacade hotelChainFacade;
    Set<String []> existingAddresses;
    Iterator<String[]> addresses;
    int nextAddress;
    String chosenHotelName = null;
    Object chosenRoomType = null;
    Date startDate = null;
    Date endDate = null;
    boolean isRoomAvailable = false;
    // ...
}
```

The code below collects the chosen hotel and room type and the dates; it checks the dates and requests the number of available rooms from the Façade. The `SimpleDateFormat` class allows us to specify the format in which dates will be supplied (allowing, for example, dates in the form 27/11/06). Dates are accessed from the appropriate text fields using the `parse` method of the `DateFormat` class, which converts the date to the form used internally by the `Date` class. If the date in the text field is in an inappropriate or incomplete form, then the `parse` method generates an exception that we have to handle appropriately.

The check out and check in times are assumed to be of 11 am and 2 pm respectively, as this helps with the calculation for the numbers of nights.

```
private void checkRoomAvailableButtonActionPerformed
    (java.awt.event.ActionEvent evt) {

    setRoomAvailable(false);
    chosenHotelName = hotelNameComboBox.getSelectedItem().toString();
    chosenRoomType = roomTypeComboBox.getSelectedItem();
    SimpleDateFormat dateFormat =
        new SimpleDateFormat("dd/MM/yy h:mm a", Locale.UK);
    boolean datesOK = true;
    try {
        endDate = dateFormat.parse(endDateField.getText() + " 11:00 AM");
    } catch (ParseException e) {
        roomFeedbackField.setText("Error in End Date");
        endDateField.selectAll();
        endDateField.requestFocus();
        endDate = null;
        datesOK = false;
    }
}
```

```

try {
    startDate = dateFormat.parse(startDateField.getText()
                                + " 2:00 PM");
} catch(ParseException e) {
    roomFeedbackField.setText("Error in Start Date");
    startDateField.selectAll();
    startDateField.requestFocus();
    startDate = null;
    datesOK = false;
}
if (datesOK) {
    if (endDate.before(startDate)) {
        roomFeedbackField.setText("End date not after start date");
        startDateField.selectAll();
        startDateField.requestFocus();
    } else {
        Date today = getTodaysDate();
        if (today != null) {
            if (startDate.before(today)) {
                roomFeedbackField.setText("Start Date before today");
                startDateField.selectAll();
                startDateField.requestFocus();
            } else {
                // find the number of rooms available of the right type
                // during these dates
                int availableRoomCount =
                    hotelChainFacade.availableRoomCount(chosenHotelName,
                                                         startDate, endDate,
                                                         chosenRoomType);

                // if any rooms available
                if (availableRoomCount > 0) {
                    // just an 's' for when available rooms is more than one
                    String plural = "s";
                    // if only one available room then no plural
                    if (availableRoomCount == 1) {
                        plural = "";
                    }
                    // feed back number of rooms available
                    roomFeedbackField.setText(availableRoomCount + " room"
                                             + plural + " available");

                    // enable the confirmation section of the screen
                    // and disable the availability section
                    setEnabledConfirmation();
                    setRoomAvailable(true);
                } else {
                    roomFeedbackField.setText("No rooms available for those dates");
                }
            }
        }
    }
}
}
}
}

```

5.2 Handling the *Confirm reservation* button of the *make reservation* use case

The following code is used as a listener on the *Confirm reservation* button. Note that *endDate*, *startDate*, *chosenHotel* and *chosenRoomType* have been previously collected by the check of available rooms. For the other fields, the code simply checks and copies data held in the appropriate components of the interface into some locally referenced objects, and calls the appropriate system method.

```

private void confirmReservationButtonActionPerformed
    (java.awt.event.ActionEvent evt) {
    String hotelName = chosenHotelName;
    Object roomType = chosenRoomType;
    String reserverName = customerNameField.getText().trim();
    String address = customerAddressTextArea.getText().trim();
    String [] multiLineAddress = address.split("\n");
    String creditCardNo = creditCardNumberField.getText().trim();

    // if both dates valid
    if (startDate == null || endDate == null ) {
        // give relevant feedback
        reservationFeedbackField.setText("Enter valid dates");
    } else if (reserverName.equals("")) {
        // give relevant feedback
        reservationFeedbackField.setText("Enter valid name");
        // allow the user to quickly edit the name field
        customerNameField.requestFocus();
    } else if (address.equals("")) {
        reservationFeedbackField.setText("Enter valid address");
        // allow the user to quickly edit the address field
        customerAddressTextArea.requestFocus();
    } else if (not a valid format for credit card (16 digits possibly with spaces)
    } else if ((existingAddresses == null) &&
        (!HotelFacade.isValidCreditCardNumberFormat(creditCardNo))) {
        // display the error text
        reservationFeedbackField.setText
            (HotelFacade.creditCardNumberFormatError(creditCardNo));
        // select all the text so that it can all be deleted with one keystroke
        creditCardNumberField.selectAll();
        // allow the user to quickly edit the field
        creditCardNumberField.requestFocus();
    } else {
        String reservationMessage;
        reservationMessage = hotelChainFacade.makeReservation(hotelName,
            startDate, endDate, roomType, reserverName, multiLineAddress,
            creditCardNo);
        reservationFeedbackField.setText(reservationMessage);
        if (reservationMessage.substring(0, 9).equals("Confirmed")) {
            // disable panel so that the reservation is now read only
            // until a new reservation is made
            setEnabledMakeReservationPanel(false);
        }
    }
}

```

5.3 Handling when a name has been entered

Once a user has entered a name we want to see if it already exists in the system and, if so, display matching addresses (in the Make Reservation panel). The user will subsequently be allowed to select from any existing matching addresses or clear the address field to enter a fresh address. We do this by associating the code below with the *Find address if existing customer* button.

```
private void findAddressButtonActionPerformed
    (java.awt.event.ActionEvent evt) {
    // name entered, does it already exist?
    String name = customerNameField.getText();
    existingAddresses = hotelChainFacade.getAddresses(name);
    addresses = existingAddresses.iterator();
    // if existing addresses exist, display the first
    if (addresses.hasNext()){
        StringBuffer displayAddress = new StringBuffer();
        String[] address = addresses.next();
        for (String addressLine : address) {
            System.out.println(addressLine);
            displayAddress.append(addressLine + "\n");
        }
        // existing addresses are not editable
        customerAddressTextArea.setEditable(false);
        customerAddressTextArea.setText(displayAddress.toString());
        // don't require re-entry of credit card details
        creditCardNumberField.setEditable(false);
        creditCardNumberField.setText("**** *");
    }
}
```

The method uses the `getAddresses()` method supplied by the hotel Façade to get any addresses for names that match this one. It then displays the first one, constructing a displayable version using a `StringBuffer` object to accumulate the lines of the address and appending new line characters. The text area where addresses are displayed is made non-editable. As we do not require re-entry of credit card details for existing customers, we make the corresponding text field non-editable and display asterisks as we do not want to display credit card details, which may well belong to some other customer (same name, different address).

5.4 Handling hotel selection

When the combo-box for selecting hotels is pressed, we want the room types displayed to match the hotel selected. We do this by providing the combo-box for room types with a model, which is an array of room types, supplied as an array of objects of `Object` type. Combo-boxes in Java use the `toString` method of an `Object` to display the string representing an object. There are two further things to note here. The Façade supplies the user interface with the room types as objects, as the interface should know very little about room types. The Swing user interface toolkit internally uses a simplified Model–View–Controller approach with the view and controller merged. The code below provides an example of this. A similar approach is used in the constructor for the user interface to supply the various combo-boxes with appropriate models.

```
private void hotelNameComboBoxActionPerformed
    (java.awt.event.ActionEvent evt) {
    ComboBoxModel rooms = new DefaultComboBoxModel
        (hotelChainFacade.getRoomTypes
            ((hotelNameComboBox.getSelectedItem()).toString()));
    roomTypeComboBox.setModel(rooms);
}
```

5.5 Handling clearing of data fields

When we move to a panel for a new use case, we need to clear the data fields, and we also want the fields used for feedback to be non-editable by the user. A reset button is used for that purpose as illustrated in the following code.

```
private void resetMakeReservationPanel() {
    existingAddresses = null;
    startDateField.setText("");
    endDateField.setText("");
    roomFeedbackField.setText("");
    customerNameField.setText("");
    reservationFeedbackField.setText("");
    creditCardNumberField.setText("");
    customerAddressTextArea.setText("");
    roomFeedbackField.setEditable(false);
    reservationFeedbackField.setEditable(false);
    setEnabledAvailability();
}
```

5.6 Summary of section

In this section we prototyped a simple user interface for the four use cases being developed.

6 Testing

We are now ready to carry out developmental testing for the completed first iteration of the first partition of our system. However, we do not have the time or space in this course to actually do all the tests; instead we will analyse what testing would be needed. We will look at enough of the testing to illustrate the principles involved. Also for reasons of time and space, we will not consider non-functional requirements.

You will probably have met most of the types of testing mentioned in this section before. Recall from *Unit 11* that developmental testing consists of unit testing, integration testing and system testing. Although only the first increment rather than the whole system is complete, it is usual with an incremental approach to perform system testing for an increment. As we are dealing with a relatively small number of classes at this stage, we will subsume integration testing under system testing. If there were some subset of classes that we were particularly concerned about we might begin with a separate integration test.

6.1 Unit testing

Each class should be submitted to both black box and white box tests for the reasons given in *Unit 11*. Owing to limited space, we will only include the tests for one class here, as an example of the testing needed.

Black box tests for class `Guest`

You should now look at the code for class `Guest`.

The class `Guest` has a single constructor and a method `equals` which we will use to illustrate testing. We will assume that the constructor is simple enough to be verified by inspecting the code. For more complex constructors, we might need some actual tests. Looking at the method `equals` we identify the tests shown in Table 1 based on the functionality expected.

Table 1 Black box tests for the method `equals` of `Guest`

Test no.	Subdomain
1	Names and addresses identical
2	Names identical, addresses different
3	Names different, addresses identical
4	Names different and addresses different

White box tests for class `Guest`

Looking at the code for the `equals` method in class `Guest` we see it has cyclomatic complexity of 5 (straight-through case, 3 `ifs`, 1 `for`). This indicates that a maximum of 5 tests will suffice to exercise each statement at least once. These are shown in Table 2.

Table 2 White box tests for the method `equals` of `Guest`

Test no.	Path	Test
1	Go straight through	Names equal, both addresses empty
2	Take first <code>if</code>	Names not equal
3	Take second <code>if</code>	Names equal, addresses with different number of lines
4	Enter first <code>for</code> loop	Names equal, addresses with same number of lines
5	Take third <code>if</code>	Names equal, addresses with same number of lines but different

Looking at tests 4 and 5 we can see that we could, in principle, use the same data. Looking a little beyond the immediate principles of testing each statement, it would seem sensible to use identical addresses in test 4, as this tests for quite an important path, even though doing so does not increase the set of statements exercised.

6.2 System testing

When undertaking system testing, we are testing the system against its requirements. Since our requirements are structured as use cases, it makes sense to structure our testing in the same way. In order to test the *cancel reservation*, *check in* and *check out* use cases, it would be extremely useful to know that the *make reservation* use case has been correctly implemented. This suggests that we should do tests based on the scenarios for individual use cases and for scenarios across the set of use cases. Indeed the error conditions being tested frequently cross use cases.

As an example, consider the case where a customer of the hotel chain makes and then cancels a reservation. If we know that the code to make the reservation is correct, we can test for cancelling the reservation as well.

Of course, even for a system of quite modest size, considering all the scenarios across a set of use cases is likely to involve a great deal of work and is worthy of automation. In order to test for use cases in isolation we will generally need to set the state, and instrument our code (for example, inserting lines of code to print values) so that we can see the effect of a scenario on the internal model of the system. We could also use JUnit.

Owing to limited space, we will illustrate system testing by first focusing on isolated tests of the *make reservation* use case. We need to test for each of the scenarios implied by our use case, and ensure that we meet the fit criteria for the associated detailed software requirements. Since in the first iteration we have chosen to focus on functionality, we will defer testing against the non-functional requirements. They will, however, need to be tested earlier rather than later in the system development, as once the architecture of the system becomes settled, any difficulties will be hard to address.

We will also test for some scenarios across the *make reservation* and *cancel reservation* use cases to illustrate inter-use-case testing, with the same considerations as above.

See <http://junit.sourceforge.net>
(Accessed 13 March 2008).

Tests for *make reservation*

The *make reservation* use case has four extension points, and so a reasonable set of tests will test the main scenario and the alternatives presented by each exception. More thorough testing might look at each permutation across the exceptions, and would be appropriate, for example, in the case of a real-time system.

We need the tests showing in Table 3.

Table 3 System tests for *make reservation*

Test no.	Scenario
1	Main success scenario
2	No availability in the hotel
3	No availability in other hotels
4	Guest already recorded

Remember that as we carry out the above tests, we will need to have selected data that shows us that the fit criteria for the detailed software requirements given in *Unit 4* are satisfied.

Tests across *make reservation* and *cancel reservation*

The *cancel reservation* use case has one main scenario and no extensions. However, we can consider the case where an attempt is made to cancel a non-existent reservation (that is, with a reservation number that is not currently allocated) as a separate scenario. An exhaustive approach to testing would combine these two scenarios in all possible combinations with the four scenarios given for the *make reservation* use case. However, we will simply use the *make reservation* use case to set us up for testing the two scenarios of *cancel reservation*. Instrumenting the model is still useful, in particular to see that the allocated room is freed up as well as the reservation itself being deleted as appropriate.

We need the tests shown in Table 4.

Table 4 System tests across *make reservation* and *cancel reservation*

Test no.	Scenario
1	Make a reservation and then cancel it
2	Cancel a non-existent reservation

6.3 Summary of section

In this section we looked briefly at unit and system testing. Some simple examples were used for illustration.

7

Summary

In this unit you have seen how to select an architecture for an application and begin its implementation with the first iteration of the first increment. You have also seen an approach to connecting an interface to a model using the Façade pattern, and some of the functionality behind a user interface. Finally, you have looked at how to select unit and system tests.

LEARNING OUTCOMES

On completion of this unit you should be able to:

- ▶ discuss application architectures;
- ▶ implement a simple model in Java;
- ▶ connect a user interface to a model using the Façade pattern;
- ▶ understand how use cases are implemented;
- ▶ outline simple unit and system tests.

References

Fowler, M. (1997) *Analysis Patterns: Reusable Object Models*, Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. and Vlissides, R. (1995) *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison Wesley.

Index

C

class methods 21

class variables 10

constructors 21

E

enum keyword 12

enumeration types 12

G

generic collection 17

I

instance methods 21

instance variables 10

M

methods 21

multiplicity 15

N

new keyword 21

null keyword 20

O

object identities 15, 20

P

predefined types 11

private keyword 10

protected keyword 10

public keyword 10

Q

qualified associations 20

Quantity pattern 12

S

static keyword 10

U

user-defined types 11

V

value identities 20

variables 10

visibility 9

